Compare std::unique_ptr, std::shared_ptr, and std::weak_ptr.

Comparison (most relevant attributes)

Attribute	`std::unique_ptr`	`std::shared_ptr`	`std::weak_ptr`
Ownership	**Exclusive**	**Shared (ref count)**	**Non-owning observer**
Copyable	No (movable only)	Yes (increments ref count)	Yes (no ref count change)
Overhead	Minimal (pointer)	Control block + atomic ops	Small (weak ref to control block)
Cycle risk	None	**Yes** (can leak)	**No** (used to break cycles)
Typical use	RAII, single owner	Shared lifetime across owners	Observe shared object safely

---

1. `std::unique_ptr` — exclusive ownership (example)

When to use: default for owning heap objects; transfer ownership with `std::move`.

```
#include <memory>
#include <iostream>

class Widget {
public:
    Widget() { std::cout << "Widget()\n"; }
    ~Widget() { std::cout << "~Widget()\n"; }
    void work() { std::cout << "working\n"; }
};
```

```
std::unique_ptr<Widget> make_widget() {
    return std::make_unique<Widget>();
}

int main() {
    auto w = make_widget();        // w owns Widget
    w->work();
    auto w2 = std::move(w);        // transfer ownership
    if (!w) std::cout << "w is null\n";
} // w2 destroyed, Widget freed
```

Key point: zero reference-count overhead; expresses exclusive ownership.

---

2. `std::shared_ptr` — shared ownership (example)

When to use: multiple owners must share lifetime. Prefer `std::make_shared` for single allocation.

```
#include <memory>
#include <iostream>

class Node {
public:
    int value;
    Node(int v) : value(v) { std::cout << "Node()\n"; }
    ~Node() { std::cout << "~Node()\n"; }
};

int main() {
    auto p1 = std::make_shared<Node>(42); // object + control block allocated together
    {
        auto p2 = p1; // ref count increments
        std::cout << "value: " << p2->value << "\n";
    } // p2 destroyed, ref count decremented
```

```
    // p1 still owns the Node
} // p1 destroyed, ref count reaches 0, Node freed
```

Key point: control block + atomic ops; use only when shared ownership is required.

---

3. `std::weak_ptr` — non-owning observer and breaking cycles

Problem: cycles with `shared_ptr` cause leaks.
Solution: make one side `weak_ptr`.

```
#include <memory>
#include <iostream>

class B; // forward

class A {
public:
    std::shared_ptr<B> b;
    ~A() { std::cout << "~A\n"; }
};

class B {
public:
    std::weak_ptr<A> a; // weak to break cycle
    ~B() { std::cout << "~B\n"; }
};

int main() {
    auto a = std::make_shared<A>();
    auto b = std::make_shared<B>();
    a->b = b;   // A owns B
    b->a = a;   // B observes A without owning it
} // both destructors run because cycle is broken
```

Using `weak_ptr` safely

```
if (auto sp = weak.lock()) {
    // sp is std::shared_ptr<T>, safe to use
} else {
    // object expired
}
```

Key point: `weak_ptr` does not extend lifetime; use `.lock()` to obtain a `shared_ptr` safely.

---

4. Common patterns, pitfalls, and remedies

- **Prefer `unique_ptr` by default.** It's the simplest and fastest.
- **Use `shared_ptr` only when ownership must be shared.** Consider redesign (ownership transfer, observer pattern) before using it.
- **Avoid cycles.** If you must have bidirectional links, make one side `weak_ptr`.
- **Use `make_shared`** to allocate object and control block together (better locality, fewer allocations).
- **Custom deleters:** both `unique_ptr` and `shared_ptr` accept custom deleters for non-`new` resources (FILE*, custom pools).

Custom deleter example (unique_ptr)

```
#include <memory>
#include <cstdio>

class FileCloser {
public:
    void operator()(FILE* f) const {
        if (f) std::fclose(f);
    }
};

int main() {
    std::unique_ptr<FILE, FileCloser> fp(std::fopen("data.txt","r"));
    // fp will call FileCloser on destruction
}
```

---

5. Thread-safety and performance notes

- **Thread-safety:** `shared_ptr`'s reference count updates are atomic (safe to copy from multiple threads), but the pointed-to object is **not** synchronized — protect shared state with mutexes or atomics.
- **Performance:** `unique_ptr` is cheapest. `shared_ptr` has higher cost (atomic increments/decrements). Avoid unnecessary `shared_ptr` copies in hot paths.
- **Deallocation cost:** `make_shared` reduces allocations; otherwise `shared_ptr` may allocate separate control block.

Example: avoid copying in hot path

void hot(std::shared_ptr<Node> p) { /* expensive copy each call */ }

// better: pass by const reference

void hot_ref(const std::shared_ptr<Node>& p) { /* no ref-count change */ }

---

6. Quick decision guide

- **Need single owner or RAII?** → `std::unique_ptr`.
- **Need shared ownership across components?** → `std::shared_ptr` + `std::make_shared`.
- **Need non-owning observer or to break cycles?** → `std::weak_ptr`.
